

SmartEye: An Open Source Framework for Real-Time Video Analytics with Edge-Cloud Collaboration

Xuezhi Wang

Nanjing University of Science and Technology
120106222717@njust.edu.cn

Guanyu Gao*

Nanjing University of Science and Technology
gygao@njust.edu.cn

ABSTRACT

Video analytics with Deep Neural Networks (DNNs) empowers many vision-based applications. However, deploying DNN models for video analytics services must address the challenges of computational capacity, service delay, and cost. Leveraging the edge-cloud collaboration to address these problems has become a growing trend. This paper provides the multimedia research community with an open source framework named SmartEye for real-time video analytics by leveraging the edge-cloud collaboration. The system consists of 1) an edge layer which enables video preprocessing, model selection, on-edge inference, and task offloading; 2) a request forwarding layer which serves as a gateway of the cloud and forwards the offloaded tasks to backend workers; and 3) a backend worker layer that processes the offloaded tasks with specified DNN models. One can easily customize the policies for preprocessing, offloading, model selection, and request forwarding. The framework can facilitate research and development in this field. The project is released as an open source project on GitHub at <https://github.com/MSNLAB/SmartEye>.

CCS CONCEPTS

• **Networks** → **Network services**; **Network architectures**; • **Computing methodologies** → *Computer vision*.

KEYWORDS

edge computing, cloud computing, video analytics, offloading

ACM Reference Format:

Xuezhi Wang and Guanyu Gao. 2021. SmartEye: An Open Source Framework for Real-Time Video Analytics with Edge-Cloud Collaboration. In *Proceedings of the 29th ACM International Conference on Multimedia (MM '21), October 20–24, 2021, Virtual Event, China*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3474085.3478330>

1 INTRODUCTION

Video analytics is widely adopted in video surveillance, autonomous vehicle, self-service supermarket to analyze the captured video content to understand environments. The state-of-the-art video analytics algorithms are mostly implemented with Deep Neural

Networks (DNNs), consisting of hundreds of layers and compute-intensive. The computational capacities of cameras and edge nodes are limited. Therefore, it may incur intolerable delays if all video analytics operations are performed locally on the cameras or edge nodes. On the other hand, it will incur substantial bandwidth costs and delays if all video contents are transmitted to the cloud for video analytics. A practical approach to improve the performances of video analytics services is to leverage the collaboration of edge and cloud for video analytics.

Leveraging the edge-cloud collaboration for video analytics can combine the advantages of edge and cloud. The edge nodes are close to local cameras, and video contents can be transmitted to the edge with lower transmission delays and less bandwidth cost. The video contents can be processed with small DNN models on the edge node or be preprocessed on the edge node to reduce video sizes and then transmitted to the cloud for processing. The cloud can provision large DNN models which have high recognition accuracy. Therefore, the cloud can support video analytics tasks that require a large processing capacity and high accuracy.

Conducting video analytics with the edge-cloud collaboration is non-trivial. The video analytics pipelines involve various processing steps [4], e.g., frame-rate and resolution tuning for video preprocessing, model selection for video analytics, on-edge processing, or offloading. Each step significantly influences the service delays and cost and must be designed appropriately by considering the processing capacities of the edge node and the cloud, the bandwidth, and the volume of transmitted video data.

The previous works optimize different components of video analytics pipelines to improve the performances. For instance, Awestream [5] adopts frame-rate and resolution tuning, and Reducto [3] sends only video frames with new objects to save bandwidth cost. VideoStorm [6] and Chameleon [2] adopt video frame sampling and resolution downsizing to save the compute cost of inferences on cheap DNN models. VideoEdge [1] designs an optimization framework to manage the tradeoff between accuracy and resource consumption for video analytics. CEVAS [7] adopts a serverless design for workload partition between edge and cloud for video analytics. These works focus on developing new mechanisms to optimize video analytics pipelines. However, researchers need to build each component of video analytics pipelines from scratch to evaluate their methods, and a flexible opensource framework is in need to facilitate research and development.

This paper presents an open source framework named SmartEye¹ for real-time video analytics by leveraging the edge-cloud collaboration. The system consists of three layers. The on-edge processing layer is implemented on the edge node. It reads video frames from the networked cameras or video files and preprocesses

*Guanyu Gao is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '21, October 20–24, 2021, Virtual Event, China

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8651-7/21/10...\$15.00

<https://doi.org/10.1145/3474085.3478330>

¹<https://github.com/MSNLAB/SmartEye>

the video frames. It also determines to adopt which server and DNN models for inference. The forwarding layer is implemented on a cloud frontend server. It accepts the offloading requests from the edge nodes and forwards the requests to the backend workers. The backend worker layer is implemented on the cloud backend servers. The workers receive the offloaded video analytics tasks from the forwarding server, perform inferences on GPU, and return the inference results. Our system is modularized and flexible. The users can easily add new components or customize the existing modules. The framework can relieve the burdens of the researchers from developing the system from scratch and make the research and development in this field more convenient.

2 SYSTEM HIGHLIGHT

Our system contains the main components of video analytics pipelines, and the users can easily set up a video analytics system with these components. The main features of our system are as follows.

Edge-cloud collaborative/standalone processing. The edge and cloud can process inference requests collaboratively. The decision engine dynamically determines whether an inference is performed on the edge node or offloaded to the cloud. Our system also supports edge/cloud standalone processing. The edge node and the cloud can perform inferences without relying on each other.

Dynamic model selection and management. The decision engine can dynamically select a model for inference based on the control policy, and dynamically load or unload modes for cache management. The model configuration file contains the offline profiling information of each model, e.g., accuracy, inference time on edge and cloud, which can facilitate decision-making.

Easy configuration of system control knobs. We provide the implementation of many commonly used video preprocessing methods, model selection policies, offloading policies, and request policies, and the users can easily configure the control knobs.

3 SYSTEM DESIGN

In this section, we first present the system architecture of SmartEye and then introduce the workflows for video analytics.

3.1 Architecture

We illustrate the system architecture of SmartEye in Fig. 1. The system mainly consists of three components: the edge node, the forwarding server, and the inference server.

Edge Node. The main functionalities of the edge node include video frame reading, video preprocessing, local inference, offloading, and decision-making. The edge node reads video frames from the camera or video files and preprocesses the video frames. After preprocessing, the inference for a video frame can be performed on the edge node or offloaded to the cloud for execution. The edge node has a decision engine to make the video preprocessing and offloading decisions based on the control policies.

Forwarding Server. The forwarding server serves as the cloud's gateway to respond to the edge node's offloading requests. When offloading an inference task, the edge node submits an HTTP request to the forwarding server by attaching the video frame. The forwarding server dispatches the inference requests to the backend servers based on the forwarding policy. The forwarding server

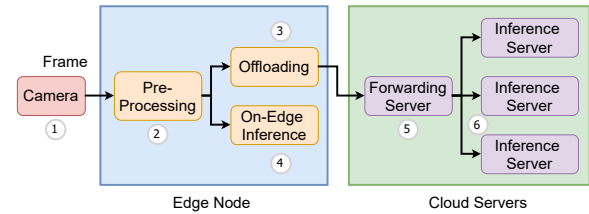


Figure 1: The system architecture of SmartEye.

also monitors the resource utilization, workload, and loaded models of each backend server and uses the backend servers' status information to make dispatch decisions.

Inference Server. The inference servers are provisioned in the cloud to conduct video analytics inferences. Each inference server loads several DNN models for video analytics. The inference servers receive the offloaded tasks from the forwarding server and make inferences with the specified models.

3.2 Workflow

The system has many control knobs, which control the workflows.

① The interval between reading two frames from the camera or the video file is configurable. The decision engine of the edge node makes decisions on ② how to preprocess a video frame, and whether the inference should be ③ offloaded to the cloud or ④ performed on the edge node, and which model should be selected. These control policies are coupled with each other because the preprocessing influences the data size, which further influences the transmission delays and inference time. ⑤ The forwarding server can make the forwarding decision based on its forwarding policy or based on the edge node's specification. ⑥ The inference server performs inferences with the specified model of the edge node.

4 SYSTEM IMPLEMENTATION

In this section, we present the detailed implementation of SmartEye. We illustrate the core modules and their interplays in Fig. 2.

4.1 On-Edge Processing Layer

Video Reader. The *Video Reader* module encapsulates the interfaces for reading video frames, and we provide three types of interfaces. The edge node can read video frames from the networked camera using Real-Time Streaming Protocol (RTSP) and read video frames from the physically connected camera as a local device. We also provide the interface to read frames from video files.

Decision Engine. The *Decision Engine* module contains the policies for video frame preprocessing, model selection, local inference, and offloading. These control actions couple with each other, and the control policy can make joint decisions. The users can also implement their policies in this module.

Pre-Processor. The *Pre-Processor* module encapsulates the functions for processing a video frame or a video file. The functions include resolution downsizing, quality tuning, frame-rate downsampling, encoding video frames into a video file. The *Pre-Processor* processes video frames under the control of *Decision Engine*.

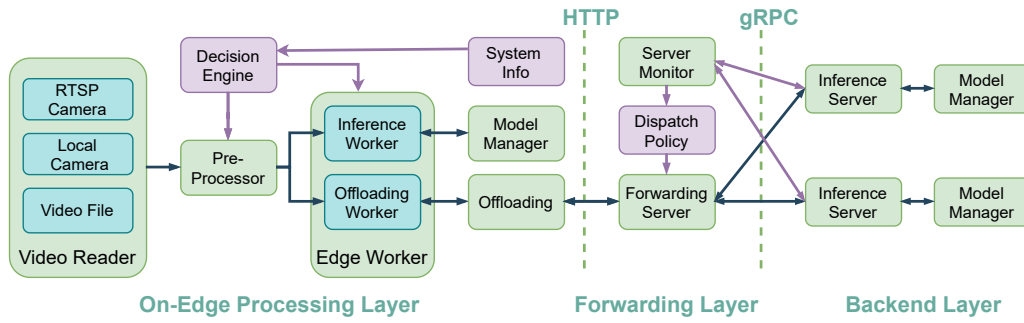


Figure 2: The implementation of the core modules in SmartEye.

Inference Worker. The *Inference Worker* is a thread that performs inferences on the edge node. It fetches the preprocessed video frames from the task queue and performs the inference for the video frames using the model selected by the *Decision Engine*. It will be blocked by the task queue when no task is available.

Offloading Worker. The *Offloading Worker* is implemented with a thread pool. An offloaded inference will be assigned to a thread in the thread pool. The thread will submit the offloading task to the forwarding server using the HTTP Post method². Whenever a new offloading task comes, it will be processed by an idle thread in the thread pool. The offloading tasks can be submitted to the cloud in parallelism to reduce queuing delays.

Model Manager. The *Model Manager* module provides the APIs to manage the DNN models for inference. Both the edge node and the backend workers use this module for model management. The functionalities include loading a set of models into memory, performing inference with a specified model, load/unload a model. The module also provides the interfaces to obtain a model's profile via reading the configuration file, for example, precision, average inference delays on the edge node, and the backend server.

Offloading. The *Offloading* module provides the interfaces to send an offloading task to the forwarding server using the HTTP Post method. The video frame sent to the forwarding server with POST is stored in the request body of the HTTP request. The selected model for inference is specified in the request.

System Information. The *System Information* module collects the edge node's resource utilization and the performances of inference and offloading, which includes CPU and memory usage, bandwidth, and processing delays. The *Decision Engine* can utilize the collected information to make control decisions.

4.2 Forwarding Layer

Forwarding Server. The *Forwarding Server* is an HTTP-based web server implemented with Flask³. It receives the offloading requests from the edge node and forwards the requests and the video frames to the backend inference servers to obtain inference results. The communications between the forwarding server and the backend inference servers are implemented with gRPC⁴.

Server Monitor. The *Server Monitor* module is implemented as a background process of the Flask server. It queries the resource utilization (e.g., CPU usage, memory usage) of each inference server under a fixed time interval. The collected information is utilized for server health monitoring and making dispatch decisions.

Dispatch Policy. The *Dispatch Policy* module provides the policies of how to dispatch the offloaded tasks among the backend inference servers. We have implemented some dispatch policies, including the shortest waiting queue, random dispatching. The users can also implement their customized dispatch policies.

4.3 Backend Layer

Inference Server. The *Inference Server* module is to process the inference requests offloaded from the edge nodes. The inference servers make inferences for video frames with the specified models and return the inference results to the forwarding server.

Model Manager. The *Model Manager* module for *Inference Server* is same with the edge node. However, it returns the performances on GPU when querying the performance of a model. Meanwhile, the inference servers and the edge nodes can load different models due to their different memory sizes and processing speeds.

5 USE CASE

We provide the use case of intelligent video surveillance to demonstrate how to build video analytics pipelines with SmartEye.

One can configure the account, password, IP address of the networked camera in the *camera-info* section of *config.ini*. The frame reading frequency and the type of video analytics inference can be specified via the command line, for example,

```
$python edge_main.py --rtsp --serv 1 -i 1000
```

The video frames will be read from the camera using RTSP, and the interval between reading two video frames is 1000ms, and object detection is specified for video analytics.

One can specify a policy for video frame preprocessing, local inference/offloading, model selection by setting the value of item *control_policy* in the *edge-setting* section of *config.ini*, e.g.,

```
control_policy=threshold_offload_policy
```

The edge node adopts *threshold_offload_policy()* as the control policy. Under this policy, if the local pending task number is less than a threshold (i.e., 2), the inference for the next video frame will be

²[https://en.wikipedia.org/wiki/POST_\(HTTP\)](https://en.wikipedia.org/wiki/POST_(HTTP))

³<https://flask.palletsprojects.com/>

⁴<https://grpc.io/>

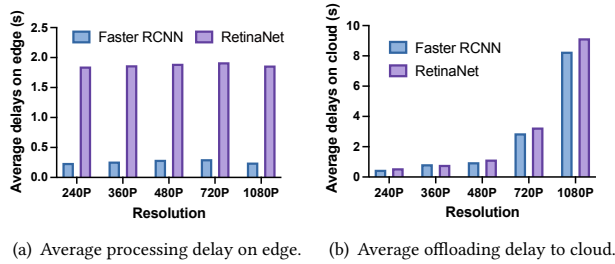


Figure 3: The average delays for inference on edge and offloading to cloud under different video resolutions.

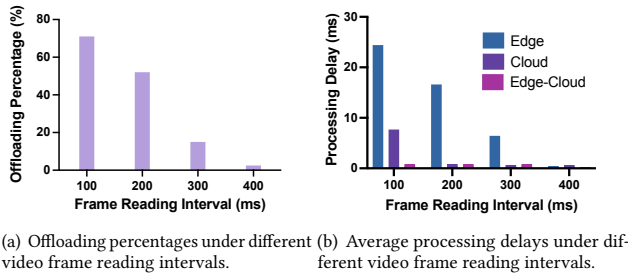


Figure 4: Performances of pure on-edge, pure offloading, and edge-cloud threshold offloading under different loads.

conducted on edge with the fastest model without resolution downsizing. Otherwise, the pre-processor downsizes the resolution of the next video frame to 240P. The edge node will offload the task to the cloud for inference with the most accurate model.

6 PERFORMANCE EVALUATION

In this section, we present the experiment settings of our testbed and evaluate the performances of our system.

Testbed. The edge node is an NVIDIA Jetson TX2⁵, with an NVIDIA Pascal GPU, 8GB memory, and an ARMv8 CPU. We use two servers as the cloud backend inference servers. Each server has 128GB memory, a GeForce RTX 2080Ti GPU, and an Intel Xeon Gold 6242R CPU. One server hosts the forwarding server. The edge node is in Jiangyin, and the cloud servers are in Nanjing. We choose object detection for video analytics and use the pre-trained models from PyTorch Hub⁶. The test video is a traffic monitoring video.

Performance influencing factors for inference on-edge and offloading to cloud. We illustrate the average processing delay using different models and video resolutions for on-edge inference and offloading to the cloud for inference in Fig. 3. The on-edge inference is very sensitive to the models' complexities and less sensitive to resolution. The average local processing delay of RetinaNet is about seven times longer than Faster RCNN. However, the processing delay for offloading to the cloud is mainly occupied by transmission time. If the video frames are downsized (e.g., 240P,

360P), the average processing delays for offloading can be significantly reduced, even smaller than on-edge inference. However, downsizing resolution comes with the cost of lower accuracy. As with several previous works [3, 4], we adopt the recognition results from a model on the original resolution (1080P) as the ground truth, and evaluate the accuracy of the model on the downsized video frames by comparing with the results from the original resolution. The COCO mAP of Faster RCNN drops to 0.752, 0.807, 0.854, 0.900, and RetinaNet drops to 0.689, 0.766, 0.813, 0.915 respectively, when the resolution is downsized to 240P, 360P, 480P, and 720P.

Performance advantages of edge-cloud collaboration for video analytics. We set the frame reading interval as different values and evaluate the average processing delays for 200 video frames. Smaller intervals lead to heavier workloads. We adopt the *threshold_offload_policy* introduced in the use case to verify the performance advantage of edge-cloud collaboration. We compared the performances with purely on-edge inference and purely offloading to the cloud with pre-downsizing to 240P on edge. The results are illustrated in Fig. 4. More inferences will be offloaded to the cloud when the loads are heavier, and the edge-cloud threshold offloading can maintain lower processing delays under heavy workloads.

7 CONCLUSION

This paper presented the implementation of an open source framework named SmartEye for real-time video analytics with the edge-cloud collaboration. The system is well implemented and easy to customize, and it can free users from the tedious works of building pipelines from scratch. We conducted some quantitative evaluations to verify the performances. We will continue to maintain SmartEye as an open source project and provide more implementations of the video processing algorithms and control algorithms.

ACKNOWLEDGMENT

This research is supported by the Key Research and Development Program of China under Grant No. 2019YFB2102200 and by Natural Science Foundation of Jiangsu Province under Grant No. BK20200486.

REFERENCES

- [1] Chien-Chun Hung, Ganesh Ananthanarayanan, Peter Bodik, Leana Golubchik, Minlan Yu, Paramvir Bahl, and Matthai Philipose. 2018. Videoeage: Processing camera streams using hierarchical clusters. In *2018 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 115–131.
- [2] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. 2018. Chameleon: scalable adaptation of video analytics. In *Proceedings of SIGCOMM 2018*. 253–266.
- [3] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 359–376.
- [4] Zhujun Xiao, Zhengxu Xia, Haitao Zheng, Ben Y Zhao, and Junchen Jiang. 2021. Towards Performance Clarity of Edge Video Analytics. *arXiv preprint arXiv:2105.08694* (2021).
- [5] Ben Zhang, Xin Jin, Sylvia Ratnasamy, John Wawrzynek, and Edward A Lee. 2018. Awstream: Adaptive wide-area streaming analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 236–252.
- [6] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J Freedman. 2017. Live video analytics at scale with approximation and delay-tolerance. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 377–392.
- [7] Miao Zhang, Fangxin Wang, Yifei Zhu, Jiangchuan Liu, and Zhi Wang. 2021. Towards cloud-edge collaborative online video analytics with fine-grained serverless pipelines. In *Proceedings of the 12th ACM Multimedia Systems Conference*. 80–93.

⁵<https://developer.nvidia.com/embedded/jetson-tx2>

⁶<https://pytorch.org/hub/>